Airline Reservation System

Midterm Project Report: Python Management System

A console-based airline reservation system built with Python and SQLite.

Project Team

Team Member	Contribution
Abduldinov Alibek	Responsible for airline.py: booking rules, input validation, search, statistics and export.
Kushekbayev Batyrlan	Responsible for main.py: console menus, user input, table display and error handling.
Aben Batyr	Responsible for database.py: SQLite connection, related tables, SQL queries and initial data generation.

Project Objective

The project provides a console application for managing flights, passengers and tickets. Users can add, view, search, update and delete records, book seats, cancel tickets and export results.

The application models reservation management. It does not process payments, connect to airline services or track refund history. Prices are stored as whole amounts in Kazakhstani tenge.

Technologies and Startup

The application requires Python and uses standard library modules: sqlite3, re, random, datetime, pathlib and argparse. No third-party packages are required to run the application. Start it with python3 main.py. The project.db database persists between runs.

Database Structure

SQLite stores data in three related tables. Each table has an id primary key. In tickets, flight_id and passenger_id are foreign keys.

Table	Fields and Purpose
flights	id, number, origin, destination, departure, seats, price. Stores the flight number, route, departure time, seat capacity and current price.
passengers	id, name, email. Stores passenger details; email is unique.
tickets	id, flight_id, passenger_id, seat, price. Stores the flight, passenger, seat number and price at booking.

Table Relationships

Relationship	Cardinality
flights.id → tickets.flight_id	One flight to many tickets
passengers.id → tickets.passenger_id	One passenger to many tickets
passengers ↔ flights through tickets	Many to many

Integrity Constraints

PRAGMA foreign_keys = ON enables relationship checks. A ticket cannot reference a nonexistent passenger or flight. UNIQUE(flight_id, seat) prevents double booking of a seat, while UNIQUE(flight_id, passenger_id) prevents a passenger from holding two tickets for the same flight. Related tickets must be cancelled before deleting a flight or passenger.

Data Sources

The dataset is synthetic and is generated by seed() or entered through the menu. The generator creates 20 flights, 50 passengers and 40 tickets only when all three tables are empty. random.Random(26) makes generation reproducible. Initial flights depart from Almaty; other origins can be entered manually. No external API is used.

After manual changes, the current database contains 20 flights, 50 passengers and 41 tickets. This meets the minimum dataset requirement of 20 / 50 / 40.

Main Functions and Classes

Module	Responsibility
main.py	Console interface: input, menus, table display and error handling.
database.py / Database	SQLite connection, table creation, SQL execution and initial data generation.
airline.py / AirlineSystem	Booking rules, validation, search, CRUD operations, statistics and export.

Input and Menus

read_text() repeats the prompt for empty input. read_number() accepts positive integers. choose_record() selects a record type. display_records() calculates column widths and prints a table. The main() loop continues until the user selects option 0.

Booking and Updates

add_ticket() calls check_booking() to verify the flight and passenger, the seat range, seat availability and duplicate bookings. The flight price is copied to the ticket. Updating a flight price does not change existing ticket prices; moving a ticket to another flight uses the new flight price. update_flight() prevents reducing capacity below the highest booked seat number.

Search and File Handling

search_passengers() searches by exact ID or part of a name; search_flights() searches by part of a destination name. The registered SQL function casefold supports case-insensitive searches, including Cyrillic text. If no matches are found, the program displays No records found. export_results() writes records and statistics to a text file.

Two Additional Features

re: regular expressions validate flight numbers, date format and email addresses. datetime.strptime() also checks calendar validity.
random: generates flight capacities and prices and selects 40 distinct passengers for initial tickets.

Example operation flow: main.py → add_ticket() → check_booking() → Database.execute() → INSERT INTO tickets → transaction commit.

Program Demonstration

The figures below render actual console output captured from main.py running on a temporary copy of project.db. They are images of captured output rather than screenshots of a terminal window. The original database was unchanged.

```
AIRLINES / Python console output
1. Passenger by name or ID
2. Flights by destination
0. Back
Choose a search: Passenger name or ID: ID |    NAME    |         EMAIL
---+------------+----------------------
1  | Alex Brown | passenger1@example.com
```

Figure 1. Searching for a passenger by name and displaying matching records.

```
AIRLINES / Python console output
1. Flight
2. Passenger
3. Ticket
0. Back
Choose a record type: Passenger name: Email: Record saved. ID: 51
```

Figure 2. Successfully adding a demonstration passenger to the database copy. Input was supplied automatically, so user responses are not echoed in the captured output.

Persistence and Export Check

After adding the demonstration passenger, the application was restarted using the same temporary database and option 7 was used to export results. The exported report contains Report Demo Student, confirming persistence between runs. This demonstration record is excluded from the original database calculations on the next page.

The defense demonstration should also cover ticket creation, rejection of an occupied seat, record updates and booking cancellation. Each team member should understand how user input leads to validation and SQL queries.

Calculations and Results

```
AIRLINES / Python console output
AIRLINE RESERVATION SYSTEM
Flights: 20
Passengers: 50
Active tickets: 41
Total seats: 1511
Total ticket revenue: 2449000 KZT
Seat occupancy rate: 2.71%
```

Figure 3. Statistics from the project database before demonstration changes.

Metric	Value
Flights / passengers / tickets	20 / 50 / 41
Total seat capacity	1511
Active ticket revenue	2 449 000 KZT
Overall seat occupancy	2.71%

Formulas

Revenue = SUM(tickets.price) = 2 449 000 KZT. Only stored active tickets are counted; cancellation deletes a ticket and reduces the total.
Occupancy = ticket count / total seat capacity × 100 = 41 / 1511 × 100 = 2.71%. If total capacity is zero, occupancy is 0.

Exported Report Contents

results.txt contains summary metrics, flights with individual occupancy rates, passengers and tickets. Statistics include all stored flights. Past departure dates are allowed; time zones are not calculated.